UD03 – Actividades voluntarias y obligatoria:

Persistencia de datos en ficheros



Unión Europea
Fondo Social Europeo
El FSE invierte en tu futuro

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, CULTURA Y DEPORTE

GENERALITAT VALENCIANA
CONSELLERIA D'EDUCACIÓ, INVESTIGACIÓ, CULTURA I ESPORT

Autor: Óscar García Escudero
Asignatura: ADA

Índice

Introducción....4
Objetivos....4
Desarrollo de las actividades....5
Actividad Obligatoria....5
Parte 1: modificando la interfaz Tkinter....6
Resultado....13
Parte 2: Funcionalidad de los botones html → obj y obj → html....14
Resultado....18
Parte 3: Funcionalidad de los botones csv → obj y obj → csv....20
Resultado....23
Parte 4: Funcionalidad de los botones json → obj y obj → json....25
Resultado....28
Parte 5: Funcionalidad de los botones obj → pickle y pickle → obj....30
Resultado....32
Parte 6: Funcionalidad de los mensajes de error a través del formato mostrado en la UI de Tkinter....34
Resultado....35
Problemas encontrados, soluciones y/o sugerencias....38
Conclusión....38
Bibliografía/webgrafía....38

Licencia



Reconeixement - No comercial – CompartirIgual(BY-NC-SA): No es permet un ús comercial de l'obra original ni de les possibles obres derivades, la distribució de les quals s'ha de fer amb una llicència igual a la que regula l'obra original.

Introducción

En esta unidad didáctica se va a trabajar la persistencia de datos a través de ficheros, como en la práctica de la unidad anterior, teniendo en cuenta la necesidad de almacenar de forma persistente la información que pueden manejar para poder utilizarla en otro momento. En concreto para esta unidad, se va a trabajar a través de la librería multiplataforma para interfaz gráfica de Python, Tkinter, el manejo de diferentes tipos de ficheros y el manejo de errores, entre otros.C

Objetivos

- Conocer la librería multiplataforma Tkinter.
- Repaso de funciones, manejo de clases y documentado con PyDoc.
- Manejo de diferentes tipos de ficheros: XML, HTML, CSV, JSON y PICKLE
- Trabajar el control de errores para que la aplicación no pase a un estado inestable devolviendo el control a la aplicación al usuario.

Desarrollo de las actividades

Repositorio de GitHub compartido: https://github.com/oscgaresc/ADA

Actividad Obligatoria

La aplicación consta de las siguientes partes:

- ***Un ventana principal de 800x600.***
- ***Un menú Archivo (ligado a la ventana principal) con las opciones:***
  - ***Abrir: permite abrir un fichero y mostrar su contenido en el editor.***
  - ***Guardar: permite guardar el contenido del editor en un archivo.***
  - ***Salir: se sale de la aplicación.***
- ***Un frame (ligado a la ventana principal) con:***
  - ***una etiqueta arriba del editor.***
  - ***un editor (ScrolledText): donde se visualizan y editan los contenidos de los ficheros.***
- ***Un frame (ligado a la ventana principal) para albergar distintos botones:***
  - ***xml->obj: permite convertir el texto XML que hay en el editor a objetos del dataclass Producto.***
  - ***obj->xml: permite convertir los objetos Producto (que al estar en el editor son texto) a un texto XML.***
  - ***html->obj: permite convertir el texto HTML que hay en el editor a objetos del dataclass Producto.***
  - ***obj->html: permite convertirlos objetos Producto (que al estar en el editor son texto) a un texto HTML.***
  - ***csv->obj: permite convertir el texto CSV que hay en el editor a objetos del dataclass Producto.***
  - ***obj->csv: permite convertirlos objetos Producto (que al estar en el editor son texto) a un texto CSV.***
  - ***json->obj: permite convertir el texto JSON que hay en el editor a objetos del dataclass Producto.***
  - ***obj->json: permite convertirlos objetos Producto (que al estar en el editor son texto) a un texto JSON.***
  - ***pickle->obj: permite convertir el texto* PICKLE (ojo que es binario) que hay en el editor a objetos del dataclass Producto.***
  - ***obj->pickle: permite convertirlos objetos Producto (que al estar en el editor son texto) a un texto* PICKLE (ojo que es binario).***

¿QUE HAY QUE HACER?

1. ***PyDoc.***
2. ***Completar la interfaz: añadir los 8 botones que faltan.***
3. ***Implementar la funcionalidad de 6 de los 8 botones que faltan. Podéis elegir. Pero de manera coherente, por ejemplo, si os dejáis pickle->obj también os dejáis su pareja obj->pickle (y viceversa).***
4. ***Hacer un control de errores exhaustivo para que el usuario tenga una experiencia agradable.***

Queda prohibido utilizar tablib y/o pandas ya que no es el propósito de esta práctica

Siguiendo todos los conceptos de la unidad y las diferentes partes que consta esta actividad obligatoria, se divide el ejercicio en varias partes, desde la creación de los botones de la interfaz hasta la funcionalidad de cada uno de los botones.

Parte 1: modificando la interfaz Tkinter

En este sentido, se procede a realizar las modificaciones necesarias, siguiendo el contendido de la documentación, de la estructura inicial del fichero:

```
# ---------------------------------------------------------------------
try:
  import Tkinter as tk  #  Es lo que indica el documento
  from Tkinter import ttk #  Es lo que indica el documento
except ImportError:
  import tkinter as tk
  from tkinter import ttk
# ---------------------------------------------------------------------
from tkinter.scrolledtext import ScrolledText
import tkinter.filedialog as tfd
import tkinter.messagebox as tmb
import xml.etree.ElementTree as ET  # librería para XML
from bs4 import BeautifulSoup       # librería para HTML
import csv                          # librería para CSV
import io                           # io.StringIO(texto) para CSV -> OBJ permite tratar
el texto como archivo
import json                         # librería para JSON
import pickle                       # librería para pickle

from functools import partial       # para poder pasar parámetros a las funciones de
callback
from dataclasses import dataclass   # almacenamiento de datos en objetos Python


# clase para guardar los datos
@dataclass
```

```
class Producto():
  genero: str
  titulo: str
  autor: str
  precio: float
  disponible: bool

  def __post_init__(self):
    self.genero = self.genero.strip() # elimina los espacios laterales
    self.titulo = self.titulo.strip() # elimina los espacios laterales
    self.autor = self.autor.strip() # elimina los espacios laterales
    self.precio = float(self.precio) # pasa de str a float
    # ¿?¿? No hace falta pasar disponible a bool ya que lo hacen las funciones
    # self.disponible = bool(self.disponible) # para que tenga todos los atributos



#
# DEFINICIÓN DE FUNCIONES
#
def seleccionar_fichero(editor):
  '''Diálogo para seleccionar un fichero y mostrarlo en el editor
  Args:
    editor(ScrolledText): zona para visualizar/editar datos
  '''
  dlg_selec = tfd.askopenfilename( # ventana para localizar ficheros
    title='Seleccionar un fichero',
    initialdir='/',
    filetypes= (('Fichero de texto', '*.txt'),
         ('Todos los ficheros', '*.*')))

  # leemos todo el fichero y se añade encoding='utf-8' al utilizar windows
  with open(dlg_selec, 'r', encoding='utf-8') as fr:
    contenido = fr.read()

  # guardamos en el editor
  editor.delete(1.0,'end')
  editor.insert('1.0', contenido)

def guardar_fichero(editor):
  '''Diálogo para guardar el contenido del editor en un fichero
  Args:
    editor(ScrolledText): zona para visualizar/editar datos
  '''
  dlg_save = tfd.asksaveasfilename( # ventana para guardar ficheros
    title="Guardar un fichero", defaultextension=".txt")
```

```
  # escribimos el fichero con el contenido del editor y se añade
  # encoding='utf-8' al utilizar windows
  with open(dlg_save, 'w', encoding='utf-8') as fw:
    fw.write(editor.get('1.0','end'))

def extraer_producto_de_xml(producto_xml):
  genero = producto_xml.attrib['genero']
  titulo = producto_xml.find('titulo').text
  autor = producto_xml.find('autor').text
  precio = producto_xml.find('precio').text
  disponible = producto_xml.find('disponible').text
  disponible = True if disponible == 'si' else False
  return Producto(genero, titulo, autor, precio, disponible)

def xml_a_objetos(editor):
  '''Parsea el texto XML y lo convierte en objetos Producto.
  Args:
    editor(ScrolledText): zona para visualizar/editar datos
  '''
  try:
    productos = []
    str_xml_editor = editor.get('1.0', 'end')

    if str_xml_editor =='\n':
      raise Exception('No hay libros o se ha cargado un xml no compatible.')

    xml = ET.fromstring(str_xml_editor)
    productos_xml = xml.findall('libro') # retorna una lista de productos
    for producto_xml in productos_xml:
      productos.append(extraer_producto_de_xml(producto_xml))
    if not productos:
      raise Exception("No hay libros o se ha cargado un xml no compatible.")

  except SyntaxError:
    tmb.showerror("Error", "Parece que la sintaxis no es XML.")
    # No se añade el icono "warning", aunque se puede probar
  except Exception as e:
    tmb.showerror("Error", f'{e}')
  else:
    # mostramos en el editor
    editor.delete('1.0','end')
    # texto_a_insertar = ''
    # for producto in productos:
    #     texto_a_insertar += str(producto) + '\n'
    # join es más eficiente que += => "separador".join(COLECCIÓN_DE_ELEMENTOS)
    texto_a_insertar = "\n".join(str(producto) for producto in productos)
```

```
    editor.insert(f'1.0',texto_a_insertar)

def objetos_a_xml(editor):
  '''genera un texto xml a partir de los objetos (en texto) que hay en el editor
  Args:
    editor(ScrolledText): zona para editar/visualizar datos
  '''
  try:
    # creamos el elemento raíz
    raiz_xml = ET.Element('libreria')

    # leemos los objetos del editor
    texto_objetos = editor.get('1.0','end')
    lista_objetos = texto_objetos.split(sep='\n')

    # quito basura
    while True:
      try:
        lista_objetos.remove('')
      except ValueError:
        break

    if not lista_objetos:
      raise Exception("No hay objetos.")

  except Exception as e:
    tmb.showerror("Error", f'{e}')

  else:
    # creamos elementos libro
    for libro in lista_objetos:
      try:
        objeto_producto = eval(libro)
      except Exception as e:
        tmb.showerror("Error", f'{e}')
      else:
        libro_xml = ET.Element('libro', attrib=dict(genero=objeto_producto.genero))
        # creamos subelementos libro y los enganchamos
        titulo_xml = ET.Element('titulo')
        titulo_xml.text = objeto_producto.titulo
        autor_xml = ET.Element('autor')
        autor_xml.text = objeto_producto.autor
        precio_xml = ET.Element('precio')
        precio_xml.text = str(objeto_producto.precio)
        disponible_xml = ET.Element('disponible')
```

```
        disponible_xml.text = "si" if objeto_producto.disponible else "no"

        libro_xml.extend([titulo_xml, autor_xml, precio_xml, disponible_xml])

        # enganchamos el libro a la raiz
        raiz_xml.append(libro_xml)

    # generamos la cadena con la raiz
    ET.indent(raiz_xml, space="\t", level=0) # prettify
    str_xml = ET.tostring(raiz_xml, encoding="unicode", xml_declaration=True)

    # mostramos en el editor
    editor.delete(1.0,'end')
    editor.insert('1.0', str_xml)



        .
        .
        .

#
# INTERFAZ GRÁFICA
#

# Configuración de la raíz (root)
root = tk.Tk()
root.geometry("800x600+50+50") # ancho x alto + x + y
root.title('Files')

# Marco para el texto
marco_texto = tk.Frame(root, height='400')
marco_texto.config(background='white')
marco_texto['borderwidth'] = 5  #Ancho del borde
marco_texto['relief'] = 'sunken' #Relieve, otros: flat, groove, raised, ridge, sunken.
marco_texto.pack(fill='x')

etiqueta_texto = ttk.Label(marco_texto, text="VISOR/EDITOR DE CONTENIDO",
            font= ("Verdana", 10),
            foreground = "white",
            background = "grey")
# etiqueta_texto.config(background='white')
etiqueta_texto.pack()

# Text + scroll
texto = ScrolledText(marco_texto)
texto.config(state='normal')
```

```
texto.pack(fill='both')

# Marco para la botonera
marco_botones = tk.Frame(root, height='150')
marco_botones.config(background='white')
marco_botones['borderwidth'] = 5
marco_botones['relief'] = 'sunken'
marco_botones.pack(fill='x',side='bottom')

# Botones que faltan de la fila 0
# botón xml -> obj
boton_xml_objetos = ttk.Button(marco_botones, text="xml->obj",
command=partial(xml_a_objetos, texto))
boton_xml_objetos.config(padding=10, width=20)
boton_xml_objetos.grid(row=0, column=0, padx=5, pady=5)

# botón html -> obj
boton_html_objetos = ttk.Button(marco_botones, text="html->obj",
command=partial(html_a_objetos, texto))
boton_html_objetos.config(padding=10, width=20)
boton_html_objetos.grid(row=0, column=1, padx=5, pady=5)

# botón csv -> obj
boton_csv_objetos = ttk.Button(marco_botones, text="csv->obj",
command=partial(csv_a_objetos, texto))
boton_csv_objetos.config(padding=10, width=20)
boton_csv_objetos.grid(row=0, column=2, padx=5, pady=5)

# botón json -> obj
boton_json_objetos = ttk.Button(marco_botones, text="json->obj",
command=partial(json_a_objetos, texto))
boton_json_objetos.config(padding=10, width=20)
boton_json_objetos.grid(row=0, column=3, padx=5, pady=5)

# botón pickle -> obj
boton_pickle_objetos = ttk.Button(marco_botones, text="pickle->obj",
command=partial(pickle_a_objetos, texto))
boton_pickle_objetos.config(padding=10, width=20)
boton_pickle_objetos.grid(row=0, column=4, padx=5, pady=5)

# Botones que faltan de la fila 1
# botón xml -> obj
boton_objetos_xml = ttk.Button(marco_botones, text="obj->xml",
command=partial(objetos_a_xml, texto))
boton_objetos_xml.config(padding=10, width=20)
boton_objetos_xml.grid(row=1, column=0, padx=5, pady=5)
```

```
# botón obj -> html
boton_objetos_html = ttk.Button(marco_botones, text="obj->html",
command=partial(objetos_a_html, texto))
boton_objetos_html.config(padding=10, width=20)
boton_objetos_html.grid(row=1, column=1, padx=5, pady=5)

# botón obj -> csv
boton_objetos_csv = ttk.Button(marco_botones, text="obj->csv",
command=partial(objetos_a_csv, texto))
boton_objetos_csv.config(padding=10, width=20)
boton_objetos_csv.grid(row=1, column=2, padx=5, pady=5)
# botón obj -> json
boton_objetos_json = ttk.Button(marco_botones, text="obj->json",
command=partial(objetos_a_json, texto))
boton_objetos_json.config(padding=10, width=20)
boton_objetos_json.grid(row=1, column=3, padx=5, pady=5)

# botón obj -> pickle
boton_objetos_pickle = ttk.Button(marco_botones, text="obj->pickle",
command=partial(objetos_a_pickle, texto))
boton_objetos_pickle.config(padding=10, width=20)
boton_objetos_pickle.grid(row=1, column=4, padx=5, pady=5)

# Creamos el menú principal
menubar = tk.Menu(root)   #Creamos el objeto Menu
root.config(menu=menubar) #Indicamos que es el menú principal

# Creamos los submenús
archivo_menu = tk.Menu(menubar, tearoff=False)

# Añadimos opciones al submenú "Archivo"
# cada opción tendrá una función o método que ejecutará
archivo_menu.add_command(label="Abrir", command=partial(seleccionar_fichero, texto)) #
label es el texto que mostrará
archivo_menu.add_command(label="Guardar", command=partial(guardar_fichero, texto))
archivo_menu.add_separator() # línea separadora
archivo_menu.add_command(label="Salir", command=root.destroy) # command es el comando
que se ejecutará

# Asignamos los submenús al menú principal
menubar.add_cascade(label="Archivo", menu=archivo_menu) # label es el texto que se
mostrará

# Bucle de la aplicación
root.mainloop()
```

A partir del código del profesor se realizan dos modificaciones en relación a la UI de Tkinter:

1. La etiqueta del texto principal: VISOR/EDITOR DE CONTENIDO donde se aplica, font, foreground y background
2. Los botones html, csv, json y pickle a objeto Producto y viceversa, tienen sus comentarios donde se diferencia cada uno de ellos

Ambas partes se pueden comprobar en el código de arriba.

Resultado

A continuación se puede ver el resultado del código anterior con los 8 botones que faltaban y la etiqueta principal del editor (no solicitada en el ejercicio):

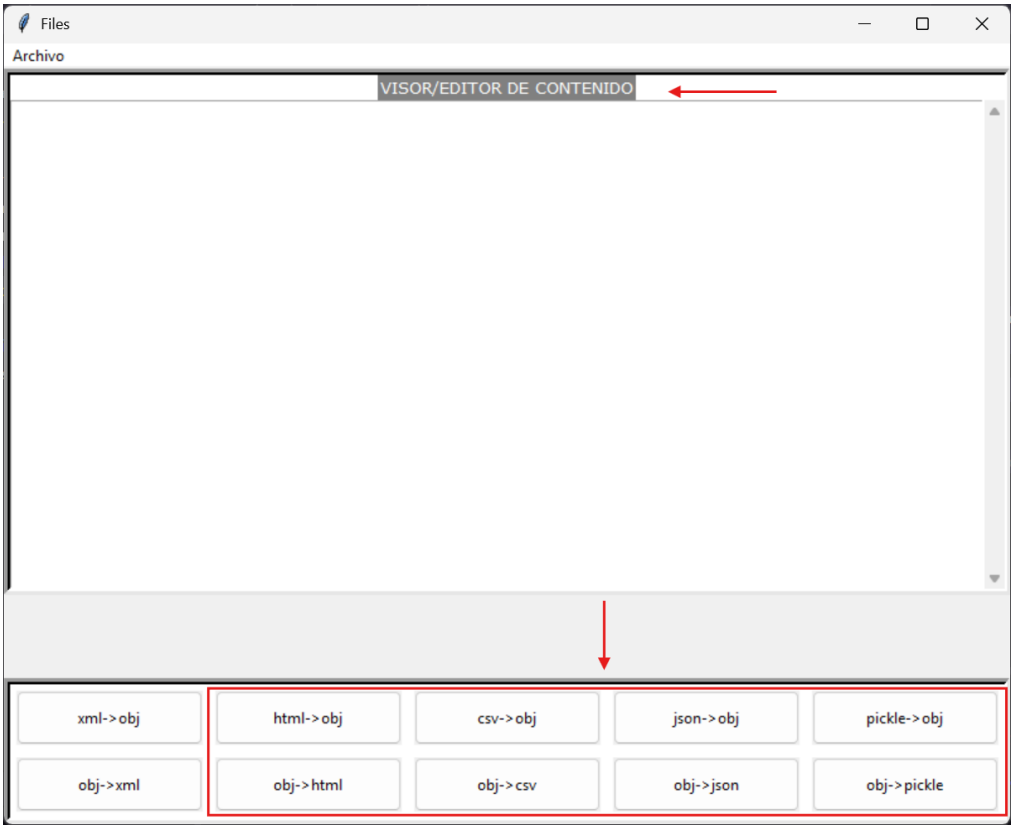


Figura 1: Interfaz tkinter de la aplicación

Teniendo en cuenta que ya está implementada la funcionalidad de los botones xml→obj y obj→xml, y además la plantilla contiene también el dataclass Producto se procede a continuar con las funcionalidades del resto de botones

Parte 2: Funcionalidad de los botones html→obj y obj→html

A continuación se añade el código con el paso a paso seguido del ejercicio a partir del documento "tips" del profesor:

```
def extraer_producto_de_html(producto_html):
  genero = producto_html.find('h2').text.strip()
  titulo = producto_html.find('div', class_='titulo').text.strip()
  autor = producto_html.find('div', class_='autor').text.strip()
  precio = producto_html.find('div', class_='precio').text.strip()
  disponible = producto_html.find('div', class_='disponible').text.strip()
  disponible = True if disponible == 'si' else False
  return Producto(genero, titulo, autor, precio, disponible)

def html_a_objetos(editor):
  '''Parsea un texto html del editor a objetos
  Args:
    editor(ScrolledText): zona para editar/visualizar datos
  '''
  try:
    # paso 1: Se obtiene texto del editor.
    productos=[]
    str_html_editor = editor.get('1.0', 'end')

    if str_html_editor =='\n':
      raise Exception('No hay libros o se ha cargado un html no compatible.')
    # paso 2: Se obtiene objeto html con BeautifulSoup con la librería lxml que es más
rápida
    html = BeautifulSoup(str_html_editor,'lxml')
    # paso 3: Se obtiene una lista de divs de clase libro (list[str])
    productos_divs_html = html.find_all('div', class_ = 'libro')
    # se obtiene una lista de strings -> (list[str]) llamada productos
    # Se utiliza .text.strip() para poder visualizar la estructura sin etiquetas
    # si lo quito me aparece las etiquetas pero se complica al utilizar los datos
    # para pasar de objeto a html en la siguiente función.
    # paso 4. Se procesa cada uno de los divs y se obtiene una lista de Productos
    for producto_html in productos_divs_html:
      productos.append(extraer_producto_de_html(producto_html))

    if not productos:
      raise Exception("No hay productos 'libros' o se ha cargado un html no
compatible.")

  except SyntaxError:
    tmb.showerror("Error", "Parece que la sintaxis no es HTML.")
    # No se añade el icono "warning", aunque se puede probar
  except Exception as e:
```

```
    tmb.showerror("Error", f'{e}')
  else:
    # paso 5. Se muestra en el editor los objetos Producto obtenidos.
    # se muestra en el editor
    editor.delete('1.0','end')

    # "separador".join(COLECCIÓN_DE_ELEMENTOS) -> 2 opciones
    texto_a_insertar = "\n".join(str(producto) for producto in productos)

    # se añade a la interfaz tkinter
    editor.insert(f'1.0',texto_a_insertar)


def html_a_objetos(editor):
  '''Parsea un texto html del editor a objetos
  Args:
    editor(ScrolledText): zona para editar/visualizar datos
  '''
  try:
    # paso 1: Se obtiene texto del editor.
    productos=[]
    str_html_editor = editor.get('1.0', 'end')

    # if str_html_editor =='\n':
    #     raise Exception('No hay libros o se ha cargado un html no compatible.')
    # Antes de pasar al siguiente bloque, con BeautifulSoup, es conveniente comprobar
    # si el html tiene un formato similar, es decir, "<" o ">".
    # Comprobando formato del editor tkinter
    formato = detecta_formato_tkinter(str_html_editor)
    if not formato =='html':
      tmb.showerror('ERROR DE FORMATO',f'Se detecta formato {formato.upper()}, no HTML')
      return # evita que recorra más código

    # paso 2: Se obtiene objeto html con BeautifulSoup con la librería lxml que es más
rápida
    html = BeautifulSoup(str_html_editor,'lxml')

    # paso 3: Se obtiene una lista de divs de clase libro (list[str])
    productos_divs_html = html.find_all('div', class_ = 'libro')
    # se obtiene una lista de strings -> (list[str]) llamada productos
    # Se utiliza .text.strip() para poder visualizar la estructura sin etiquetas
    # si lo quito me aparece las etiquetas pero se complica al utilizar los datos
    # para pasar de objeto a html en la siguiente función.
    # paso 4. Se procesa cada uno de los divs y se obtiene una lista de Productos
    for producto_html in productos_divs_html:
      productos.append(extraer_producto_de_html(producto_html))
```

```
    if not productos:
      raise Exception("No hay libros o se ha cargado un html no compatible.")

  except SyntaxError:
    tmb.showerror("Error", "Parece que la sintaxis no es HTML.")
    # No se añade el icono "warning", aunque se puede probar
  except Exception as e:
    tmb.showerror("Error", f'{e}')
  else:
    # paso 5. Se muestra en el editor los objetos Producto obtenidos.
    # se muestra en el editor
    editor.delete('1.0','end')

    # "separador".join(COLECCIÓN_DE_ELEMENTOS) -> 2 opciones
    texto_a_insertar = "\n".join(str(producto) for producto in productos)

    # se añade a la interfaz tkinter
    editor.insert(f'1.0',texto_a_insertar)


def objetos_a_html(editor):
  '''genera un texto html a partir de los objetos (en texto) que hay en el editor
  Args:
    editor(ScrolledText): zona para editar/visualizar datos
  '''
  try:
    # paso 1. Se obtiene el texto del editor
    texto_objetos = editor.get('1.0','end-1c') # 1c para evitar el \n final
    # de Tkinter, que da problemas

    # Comprobando formato del editor tkinter
    formato = detecta_formato_tkinter(texto_objetos)

    if not formato =='objeto Producto':
      tmb.showerror('ERROR DE FORMATO',f'Se detecta formato {formato.upper()}, no OBJETO
PRODUCTO')
      return # evita que recorra más código

    # paso 2. Se obtiene una lista de texto_objetos separando por \n
    lista_objetos = texto_objetos.split(sep='\n')

    # paso 3. Se quita basura ('')
    while True:
      try:
        lista_objetos.remove('')
```

```
    except ValueError:
      break

  if not lista_objetos:
    raise Exception("No hay objetos.")

except Exception as e:
  tmb.showerror("Error", f'{e}')

else:
  # paso 4. Se crea el documento HTML con beautifulsoup
  soup = BeautifulSoup('<html><head></head><body></body><footer>'\
            '</footer></html>', 'html.parser')
  title = soup.new_tag('title')
  title.string = 'Libros de la librería'
  soup.head.append(title)
  # los css con style
  style = soup.new_tag('style')
  style.string = '''body {text-align: center;}
  h1 {color: grey;}
  h2 {color: navy;}
  .precio {font-weight: bold}
  footer {background: gray; color: white; margin: 20px;padding: 20px;}
  '''
  soup.head.append(style)
  h1 =soup.new_tag('h1')
  h1.string = 'librería'
  soup.body.append(h1)
  #-----------------------------------------------------------------------
  #                  AÑADIENDO LOS OBJETOS A HTML
  #-----------------------------------------------------------------------
  # Se añaden los objetos del editor y se transforma a html y se crea
  # elementos libro
  for libro in lista_objetos:
    # Limpiando espacios en blanco para evitar errores
    libro = libro.strip()
    try:
      objeto_producto = eval(libro) # se obtiene el objeto producto
      # eval(), evalúa cualquier tipo de dato, por lo que es conveniente comprobar si
      # el tipo de dato es de tipo Producto de la siguiente manera:
      if not isinstance(objeto_producto, Producto):
        raise ValueError(f'El elemento {libro} no es un objeto Producto válido.')

    except Exception as e:
      tmb.showerror("Error", f'Línea inválida, no representa un objeto Producto'+
             f' válido:\n{libro}\n\n{e}')
```

```
      return # se cierra la función sin que realice cambios para mejorar la ux
    else:
      div_libro =soup.new_tag('div', attrs={'class':'libro'})
      h2_genero =soup.new_tag('h2')
      h2_genero.string = str(objeto_producto.genero)
      div_libro.append(h2_genero)
      div_titulo =soup.new_tag('div', attrs={'class':'titulo'})
      div_titulo.string = str(objeto_producto.titulo)
      div_libro.append(div_titulo)
      div_autor =soup.new_tag('div', attrs={'class':'autor'})
      div_autor.string = str(objeto_producto.autor)
      div_libro.append(div_autor)
      div_precio =soup.new_tag('div', attrs={'class':'precio'})
      div_precio.string = str(objeto_producto.precio)
      div_libro.append(div_precio)
      div_disponible =soup.new_tag('div', attrs={'class':'disponible'})
      div_disponible.string = 'si' if objeto_producto.disponible else 'no'
      div_libro.append(div_disponible)

      soup.body.append(div_libro)
      #-----------------------------------------------------------------------

soup.footer.string='Me gusta Python'

# Se configura la identación con el metodo prettyfy()
texto_html = soup.prettify()

# paso 6. Se muestra en el editor
editor.delete(1.0,'end')
editor.insert('1.0', texto_html)
```

Como se puede observar al principio del código se crea una nueva función llamada extraer_producto_de_html() que, como indica su nombre, crea un objeto Producto a partir de fichero CSV recodigo del editor. Con este código se consigue el siguiente resultado.

Resultado

Tras realizar pruebas del funcionamiento del código anterior se obtiene los siguientes resultados.

- Abriendo el fichero *libro.html y mostrándolo en la pantalla del editor.*

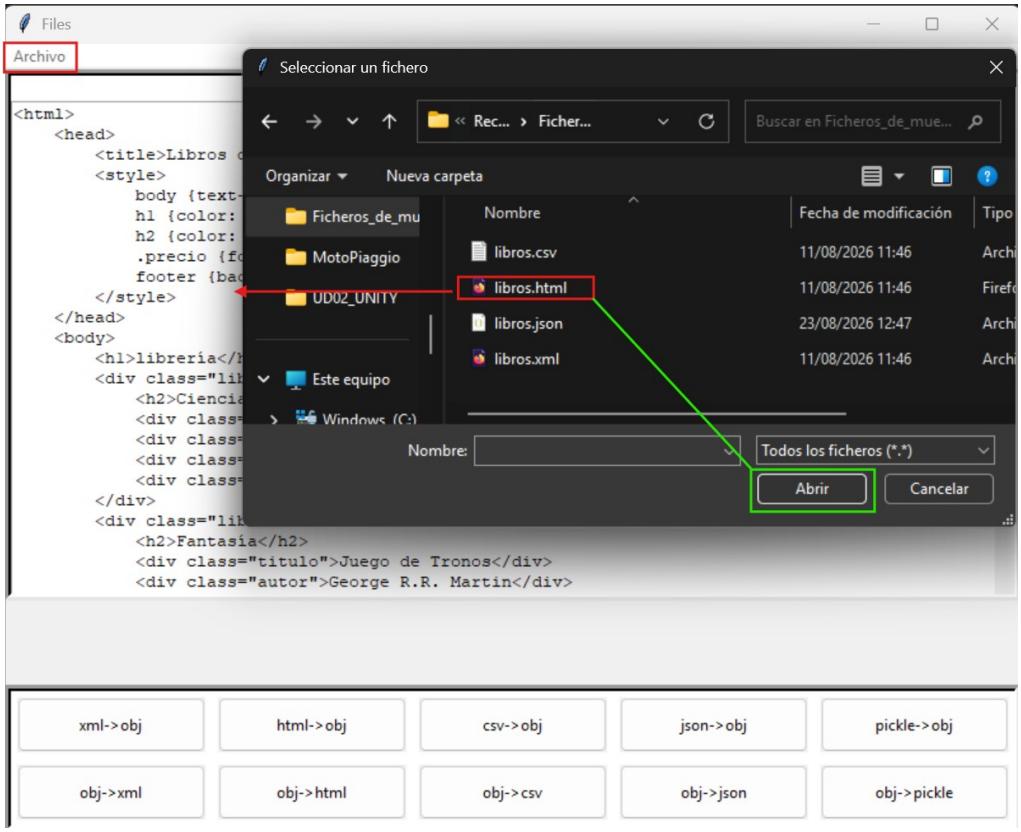


Figura 2: Mostrando por pantalla el fichero libro.html

- Pasando de *html* a objeto de la clase *Producto* con el botón *html→obj*.

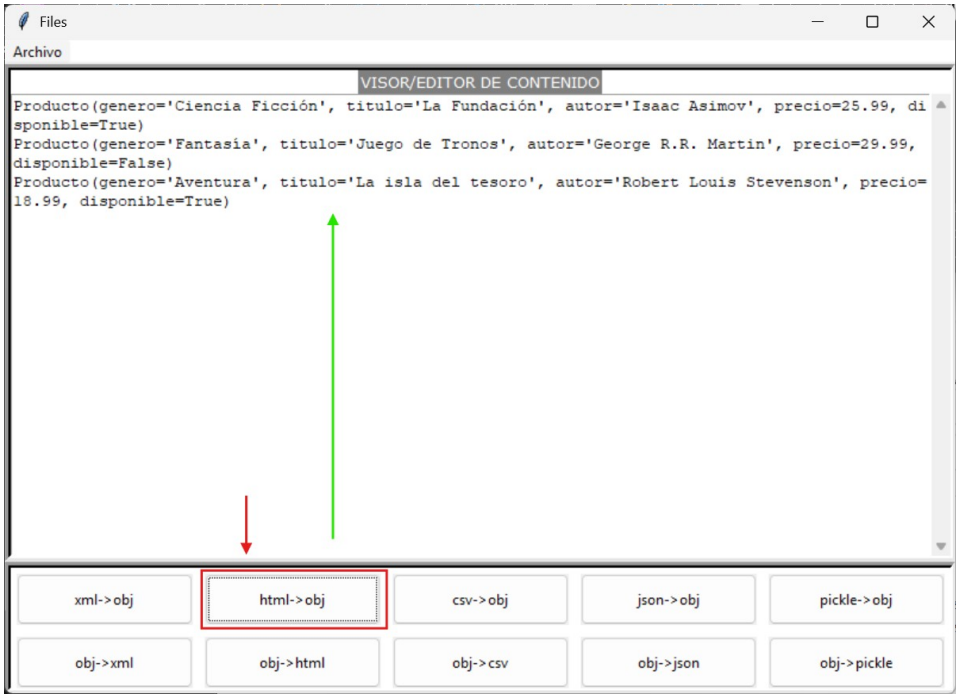


Figura 3: Mostrando objetos de la clase Productos a partir de un fichero html

- *Volviendo de objeto de la clase Producto a html con el botón obj→html.*

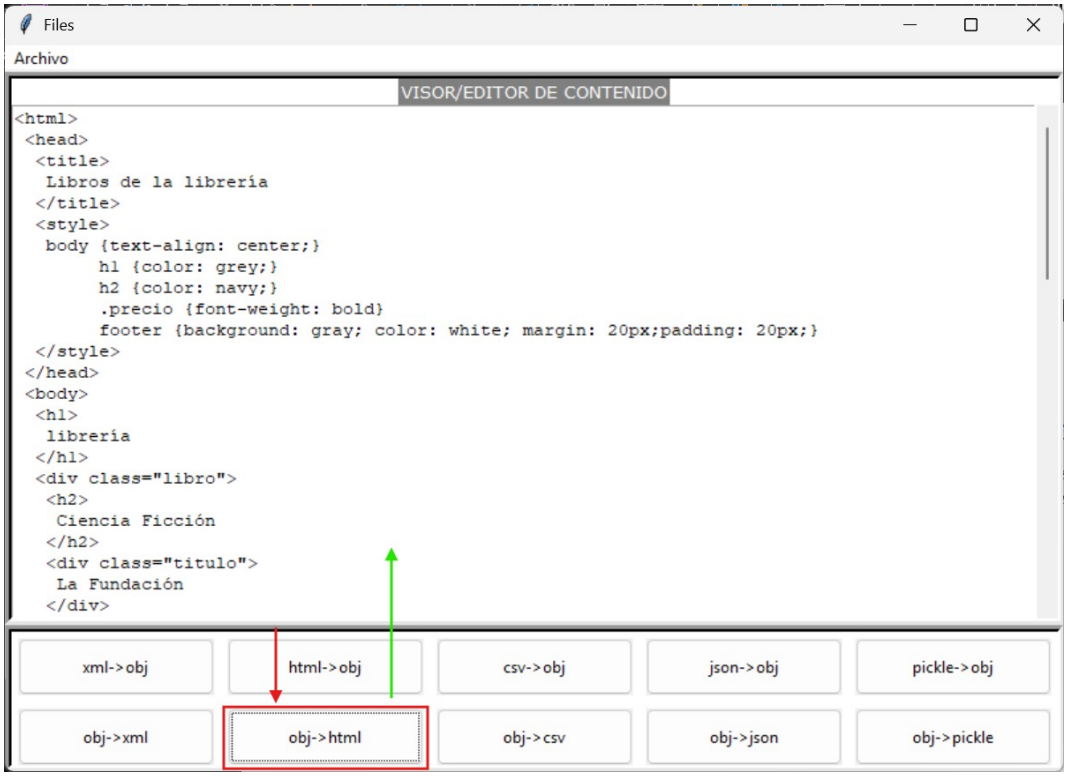


Figura 4: Mostrando html a partir del contendido de los objetos de la clase Producto.

Aunque no se siga repitiendo el resto de acciones, se puede volver a realizar la fase anterior para volver a mostrar los objetos de la clase Producto con el botón *html→obj*.

Parte 3: Funcionalidad de los botones csv→obj y obj→csv

A continuación se añade el código con el paso a paso del ejercicio seguido a partir del documento "tips" del profesor, para este segundo par de botones:

```
def extraer_producto_de_csv(producto_csv):
  genero, titulo, autor, precio, disponible = producto_csv.split(',')
  disponible = True if disponible == 'si' else False
  return Producto(genero, titulo, autor, precio, disponible)

def csv_a_objetos(editor):
  '''Parsea un texto csv del editor a objetos
  Args:
    editor(ScrolledText): zona para editar/visualizar datos
  '''
  try:
    # paso 1: Se obtiene texto del editor.
    productos=[]
    str_csv_editor = editor.get('1.0', 'end-1c')

    # if str_csv_editor =='\n':
    #     raise Exception('No hay libros o se ha cargado un csv no compatible.')
```

```
    # Comprobando formato del editor tkinter
    formato = detecta_formato_tkinter(str_csv_editor)
    if not formato =='csv':
      tmb.showerror('ERROR DE FORMATO',f'Se detecta formato {formato.upper()}, no CSV')
      return # evita que recorra más código

    # Se comprueba si el csv (str_csv_editor) tiene cabecera
    tiene_cabecera = csv.Sniffer().has_header(str_csv_editor)

    # paso 2. Se convierte el texto_csv en lista_cadenas (split(sep='\n')) eliminando
vacíos
    lista_objetos = str_csv_editor.split(sep='\n')
    # Lista de cadena sin vacíos
    lista_cadenas = [cadena_csv for cadena_csv in lista_objetos if cadena_csv.strip()]

    # paso 3: Se crea un iterador_lista_cadenas con iter(lista_cadenas)
    iterador_lista_cadenas = iter(lista_cadenas)
    # paso 4: Se genera la lista de productos con los objetos Producto del csv
    if tiene_cabecera:
      next(iterador_lista_cadenas) # ignora la cabecera

    for producto_csv in iterador_lista_cadenas:
      #yield Producto(*fila) # descomponemos la list
      productos.append(extraer_producto_de_csv(producto_csv))

    if not productos:
      raise Exception("No hay productos 'libros' o se ha cargado"\
             " un csv no compatible.")

  except SyntaxError:
    tmb.showerror("Error", "Parece que la sintaxis no es CSV.")
    # No se añade el icono "warning", aunque se puede probar
  except Exception as e:
    tmb.showerror("Error", f'{e}')
  else:
    # paso 5. Se muestra en el editor los objetos Producto obtenidos.
    # se muestra en el editor
    editor.delete('1.0','end')

    # "separador".join(COLECCIÓN_DE_ELEMENTOS) -> 2 opciones
    texto_a_insertar = '\n'.join(str(producto) for producto in productos)

    # se añade a la interfaz tkinter
    editor.insert(f'1.0',texto_a_insertar)
```

```
def objetos_a_csv(editor):
  '''genera un texto csv a partir de los objetos (en texto) que hay en el editor
  Args:
    editor(ScrolledText): zona para editar/visualizar datos
  '''
  try:
    # paso 1. Se obtiene el texto del editor
    texto_objetos = editor.get('1.0','end-1c') # 1c para evitar el \n final
    # de Tkinter, que da problemas

    # Comprobando formato del editor tkinter
    formato = detecta_formato_tkinter(texto_objetos)
    if not formato =='objeto Producto':
      tmb.showerror('ERROR DE FORMATO',f'Se detecta formato {formato.upper()}, no OBJETO
PRODUCTO')
      return # evita que recorra más código

    # paso 2. Se obtiene una lista de texto_objetos separando por \n
    lista_objetos = texto_objetos.split(sep='\n')

    # paso 3. Se quita basura ('')
    while True:
      try:
        lista_objetos.remove('')
      except ValueError:
        break

    if not lista_objetos:
      raise Exception("No hay objetos.")

  except Exception as e:
    tmb.showerror("Error", f'{e}')

  else:
    # paso 4: Se crea la cabecera en productos_csv
    cabecera = ['genero','titulo','autor','precio','disponible']
    productos_csv = ['genero, titulo, autor, precio, disponible']

    # paso 5: Se pasa el contenido del editor a objetos Productos con el
    # método eval para usarlo para generar el csv
    for libro in lista_objetos:
      # Limpiando espacios en blanco para evitar errores
      libro = libro.strip()
      try:
        prod = eval(libro) # se obtiene el objeto producto
      except Exception as e:
```

```
            tmb.showerror("Error", f'{e}')
        else:
            disponible = 'si' if prod.disponible == True else 'no'
            linea_csv = f'{prod.genero},{prod.titulo},{prod.autor},'\
              f'{str(prod.precio)},{disponible}'
            productos_csv.append(linea_csv)

    # mostramos en el editor
    editor.delete('1.0','end')

    # join es más eficiente que += => "separador".join(COLECCIÓN_DE_ELEMENTOS)
    texto_a_insertar = "\n".join(str(producto) for producto in productos_csv)

    editor.insert(f'1.0',texto_a_insertar)
```

Como se puede observar al principio del código se crea una nueva función llamada extraer_producto_de_csv() que como indica su nombre crea un objeto Producto a partir de fichero CSV recogido del editor. Como en el caso anterior, se realiza el mismo paso a paso hasta conseguir el siguiente resultado.

Resultado

Abriendo el fichero *libro.csv y mostrándolo en la pantalla del editor.*

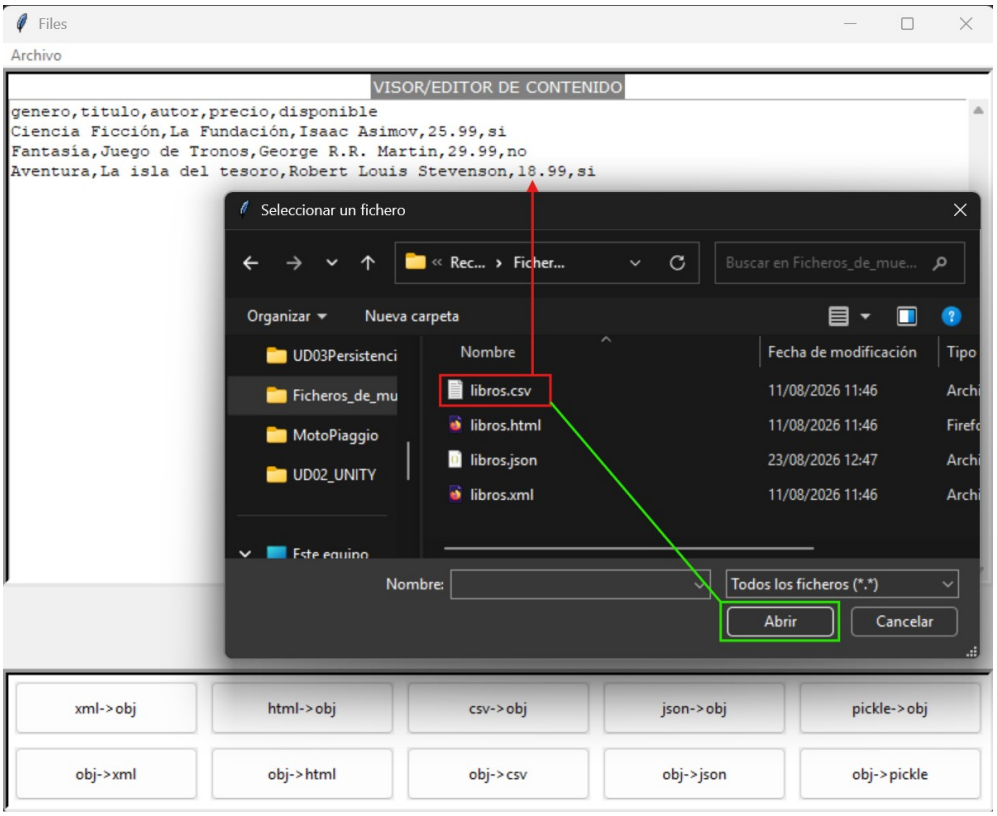


Figura 5: Mostrando por pantalla el fichero libro.csv

- Pasando de *csv* a objeto de la clase *Producto* con el botón *csv → obj*.

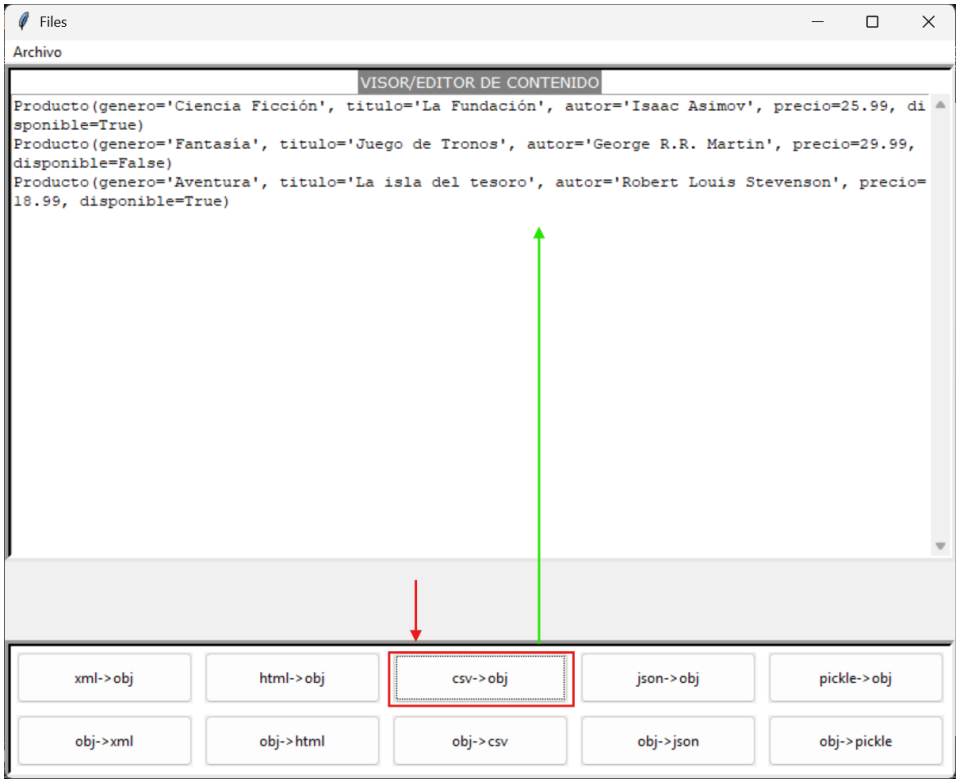


Figura 6: Mostrando objetos de la clase Productos a partir de un fichero csv

- *Volviendo de objeto de la clase Producto a csv con el botón obj → csv.*

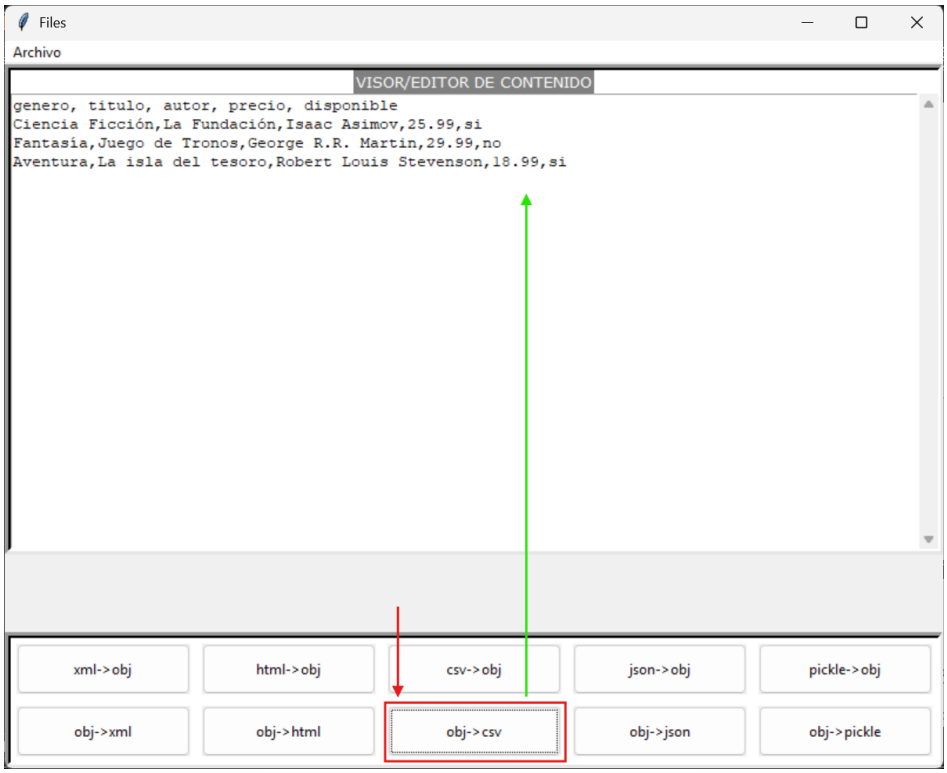


Figura 7: Mostrando csv a partir del contendido de los objetos de la clase Producto.

Aunque no se siga repitiendo el resto de acciones, se puede volver a realizar la fase anterior para volver a mostrar los objetos de la clase Producto con el botón *csv→obj*, como sucedió en el caso anterior.

Parte 4: Funcionalidad de los botones json→obj y obj→json

A continuación se añade el código con el paso a paso del ejercicio seguido a partir del documento "tips" del profesor, para este tercer par de botones:

```
def como_productos(dct):
  '''Retorna un objeto Producto a partir de un diccionario
  Args:
    dct(dict): diccionario con la información necesaria para
    crear un objeto Producto
  '''
  return Producto(dct['genero'], dct['titulo'], dct['autor'],
         dct['precio'], dct['disponible'])

class ProductoEncoder(json.JSONEncoder):
  '''Si el objeto es de tipo Producto lo pasa a dict para que
  json.JSONEncoder pueda codificarlo.
  En otro caso, json.JOSNEncoder lo codifica directamente
  '''
  def default(self, obj):
    if isinstance(obj, Producto):
      return dict(genero=obj.genero, titulo=obj.titulo, autor=obj.autor,
            precio=float(obj.precio), disponible=obj.disponible)
    return json.JSONEncoder.default(self, obj)


def json_a_objetos(editor):
  '''Parsea un texto json del editor a objetos
  Args:
    editor(ScrolledText): zona para editar/visualizar datos
  '''
  try:
    # paso 1: Se obtiene texto del editor.
    productos=[]
    str_json_editor = editor.get('1.0', 'end-1c')

    # if str_json_editor =='\n':
    #     raise Exception('No hay libros o se ha cargado un json no compatible.')

    # Comprobando formato del editor tkinter
    formato = detecta_formato_tkinter(str_json_editor)
```

```
    if not formato =='json':
      tmb.showerror('ERROR DE FORMATO',f'Se detecta formato {formato.upper()}, no
JSON')
      return # evita que recorra más código

    # paso 2. Se obtiene una lista de Productos mediante json.loads, usando
como_productos
    # para indicar como obtener un producto a partir de un diccionario JSON
    # se utiliza json.loads, con "s", para cargar una cadena de texto no un texto
abierto,
    # además la cadena de texto debe contener todo el json en un string
    productos = json.loads(str_json_editor, object_hook=como_productos)

    if not productos:
      raise Exception("No hay productos 'libros' o se ha cargado"\
              " un json no compatible.")

  except SyntaxError:
    tmb.showerror("Error", "Parece que la sintaxis no es JSON.")
    # No se añade el icono "warning", aunque se puede probar
  except Exception as e:
    tmb.showerror("Error", f'{e}')
  else:
    # paso 3. Se muestra en el editor los objetos Producto obtenidos.
    # se muestra en el editor
    editor.delete('1.0','end') # primero se borra

    # "separador".join(COLECCIÓN_DE_ELEMENTOS) -> 2 opciones
    texto_a_insertar = '\n'.join(str(producto) for producto in productos)

    # se añade los productos a la interfaz tkinter
    editor.insert(f'1.0',texto_a_insertar)


def objetos_a_json(editor):
  '''genera un texto json a partir de los objetos (en texto) que hay en el editor
  Args:
    editor(ScrolledText): zona para editar/visualizar datos
  '''
  try:
    # paso 1. Se obtiene el texto del editor
    texto_objetos = editor.get('1.0','end-1c') # 1c para evitar el \n final
    # de Tkinter, que da problemas

    # Comprobando formato del editor tkinter
    formato = detecta_formato_tkinter(texto_objetos)
```

```
    if not formato =='objeto Producto':
      tmb.showerror('ERROR DE FORMATO',f'Se detecta formato {formato.upper()}, no
OBJETO PRODUCTO')
      return # evita que recorra más código

    # paso 2. Se obtiene una lista de texto_objetos separando por \n
    lista_objetos = texto_objetos.split(sep='\n')

    # paso 3. Se quita basura ('')
    while True:
      try:
        lista_objetos.remove('')
      except ValueError:
        break

    if not lista_objetos:
      raise Exception("No hay objetos.")

  except Exception as e:
    tmb.showerror("Error", f'{e}')

  else:
    # paso 4: Se crea una variable productos_json inicializada a '[' eliminada
    # se crea directamente una lista
    lista_productos = []
    # productos_json.append('[') # inicio del json
    # paso 5: Se pasa el contenido del editor a objetos Productos con el
    # método eval para usarlo para completar la lista de productos
    for libro in lista_objetos:
      # Limpiando espacios en blanco para evitar errores
      libro = libro.strip()
      try:
        prod = eval(libro) # se obtiene el objeto producto
      except Exception as e:
        tmb.showerror("Error", f'{e}')
      else:
        # Añado todos los productos a una lista de productos_json
        lista_productos.append(prod)

    # Finalmente no se hace uso de __dict__
    # Se añade directamente la lista de productos en json.dumps()
    # 1. Se añade la lista a realizar dumps lista_productos
    # 2. cls apunta a la funcion ProductoEncoder si es objeto o
    # se ha pasado a diccionario
    # 3. indent que crea una indentación de 4 espacios
    # 4. ensure_ascii = False, para que no haya problemas con los caracteres
```

```
    # especiales del castellano
    productos_json = json.dumps(lista_productos, cls=ProductoEncoder, indent=4,
                  ensure_ascii=False)
    # De esta manera se realiza un único dumps para que cree los corchetes y
    # las comas necesarias, evitando realizarlo de manera manual con un for
    # por productos y realizar manualmente los corchetes y las comas entre
    # productos.

    # mostramos en el editor
    editor.delete('1.0','end')

    editor.insert(f'1.0', productos_json)
```

Como se puede observar al principio del código aparecen las funciones como_producto(dict), que facilita la transformación de json a objeto a través del método json.loads() de la librería json, y ProductoEncoder(json.JSONEncoder), que realizar la transformación de objeto a json a través de método json.dumps(). Estas funciones ya estaban en el código, así que solo ha sido necesario aplicarlas en las funciones correspondientes. Como en el caso anterior, se realiza el mismo paso a paso hasta conseguir el siguiente resultado.

Resultado

Abriendo el fichero *libro.json y mostrándolo en la pantalla del editor.*

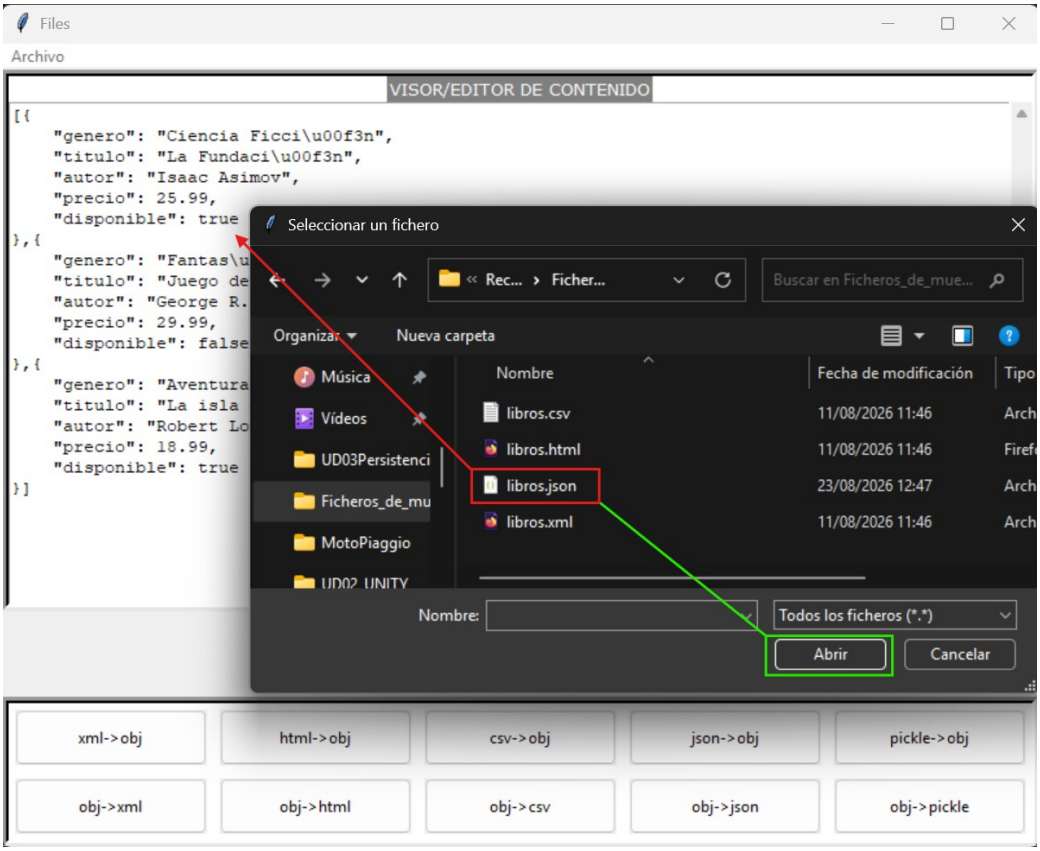


Figura 8: Mostrando por pantalla el fichero libro.json

- Pasando de *json* a objeto de la clase *Producto* con el botón *json→obj*.

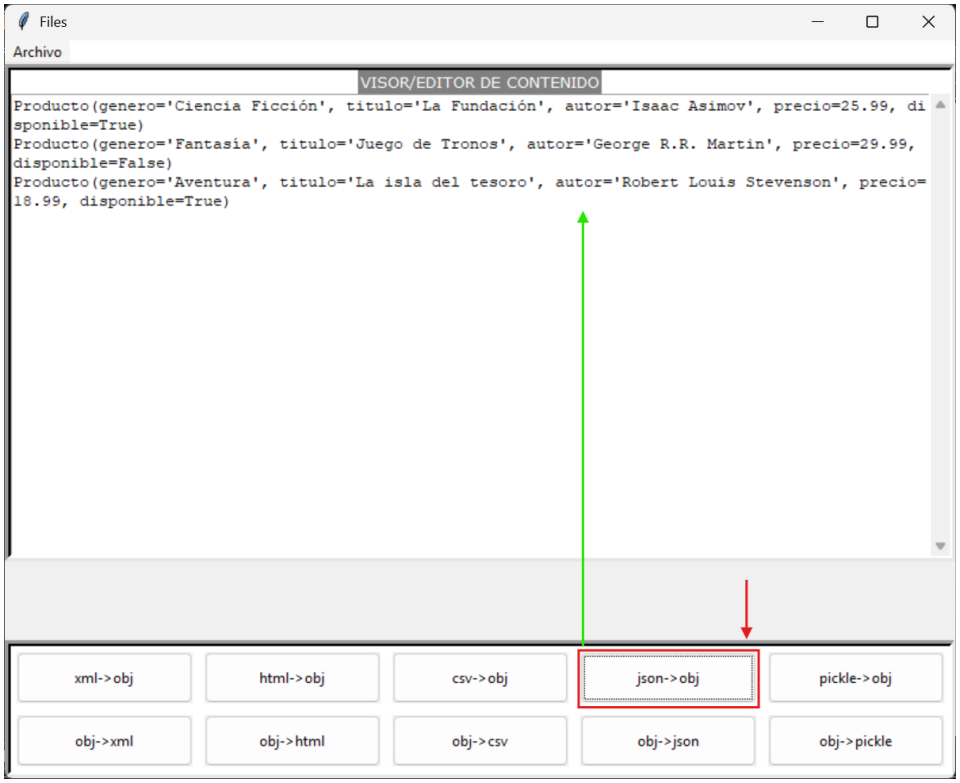


Figura 9: Mostrando objetos de la clase Productos a partir de un fichero json

- *Volviendo de objeto de la clase Producto a json con el botón obj→json.*

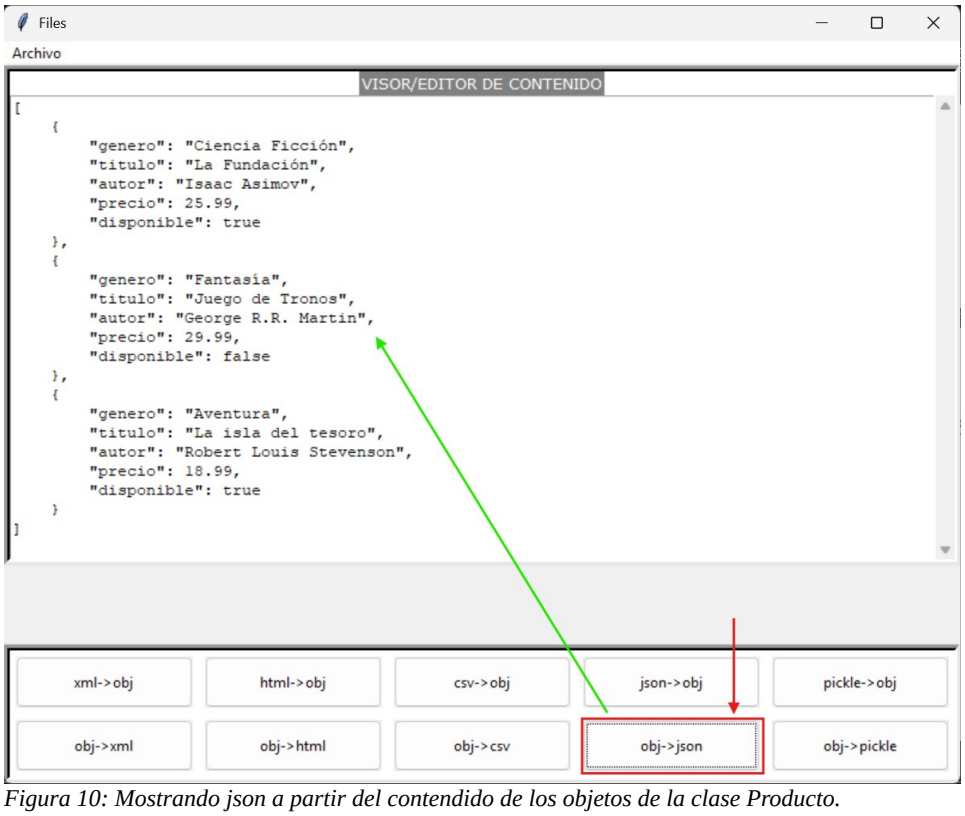


Figura 10: Mostrando json a partir del contendido de los objetos de la clase Producto.

Aunque no se siga repitiendo el resto de acciones, se puede volver a realizar la fase anterior para volver a mostrar los objetos de la clase Producto con el botón *json→obj*.

Parte 5: Funcionalidad de los botones obj→pickle y pickle→obj

A continuación se añade el código con el paso a paso del ejercicio seguido a partir del documento "tips" del profesor, para este último par de botones:

```
# VARIABLE GLOBAL
lista_objetos_pickle = []

def pickle_a_objetos(editor): # VUELTA: Se debe ejecutar con el pickle en el editor
  '''deserializa los objetos pickle del editor a objetos Python (muestra el resultado
en el editor).
  En la variable global lista_objetos_pickle está el resultado de la serialización
anterior.

  Args:
    editor(ScrolledText): zona para editar/visualizar datos
  '''
  try:
    # Se tiene en cuenta que los objetos_pickle (byte) están en la variable global
    global lista_objetos_pickle
    lista_productos=[] # se inicializa la lista

    # Comprobando formato del editor tkinter
    str_pickle_editor = editor.get('1.0', 'end-1c')
    formato = detecta_formato_tkinter(str_pickle_editor)
    if not formato =='pickle':
      tmb.showerror('ERROR DE FORMATO',f'Se detecta formato {formato.upper()}, no
PICKLE')
      return # evita que recorra más código

    # Comprobando si hay lista de objetos_pickle, segunda defensa...
    if not lista_objetos_pickle:
      raise Exception("No hay datos en la variable global.")

    # No se recogen los datos del editor como hasta ahora
    # paso 2. Se añade el contenido del editor a la lista de objetos mediante
    # pickle.loads
    lista_productos = pickle.loads(lista_objetos_pickle)

    if not lista_productos:
        raise Exception("No hay productos 'libros' o se ha cargado"\
               " un pickle no compatible.")

  except SyntaxError:
```

```
    tmb.showerror("Error", "Parece que la sintaxis no es PICKLE.")
    # No se añade el icono "warning", aunque se puede probar
  except Exception as e:
    tmb.showerror("Error", f'{e}')
  else:
    # paso 3. Se muestra en el editor los objetos Producto obtenidos.
    # se muestra en el editor
    editor.delete('1.0','end') # primero se borra

    # "separador".join(COLECCIÓN_DE_ELEMENTOS) -> 2 opciones
    texto_a_insertar = '\n'.join(str(producto) for producto in lista_productos)

    # se añade los productos a la interfaz tkinter
    editor.insert(f'1.0',texto_a_insertar)
    print(str(lista_objetos_pickle))

def objetos_a_pickle(editor): # IDA: Se debe ejecutar con los productos en el editor
  '''serializa los objetos del editor a pickle (muestra el resultado en el editor)
  Al mismo tiempo se guarda una variable global lista_objetos_pickle con todas las
serializaciones.
  Esto es porque pickle solo serializa un objeto cada vez
  Args:
    editor(ScrolledText): zona para editar/visualizar datos
  '''
  try:
    # paso 1. Se vacia la variable global lista_objetos_pickle
    global lista_objetos_pickle # para trabajar dentro de la función con la variable
global
    lista_objetos_pickle = [] # No utilizo clear porque pasa a bytes y no existe

    # paso 2. Se obtiene el texto del editor y se comprueba su formato
    texto_objetos = editor.get('1.0','end-1c') # 1c para evitar el \n final
    formato = detecta_formato_tkinter(texto_objetos)
    if not formato =='objeto Producto':
      tmb.showerror('ERROR DE FORMATO',f'Se detecta formato {formato.upper()}, no OBJETO
PRODUCTO')
      return # evita que recorra más código

    # paso 3. Se obtiene una lista de texto_objetos separando por "\n"
    lista_objetos = texto_objetos.split(sep='\n')

    # paso 3. Se quita basura ('')
    while True:
      try:
        lista_objetos.remove('')
      except ValueError:
```

```
      break

  if not lista_objetos:
    raise Exception("No hay objetos.")

except Exception as e:
  tmb.showerror("Error", f'{e}')

else:
  # paso 4: Se crea una variable productos_json inicializada a '[' eliminada
  # se crea directamente una lista
  lista_productos = []
  # productos_json.append('[') # inicio del json
  # paso 5: Se pasa el contenido del editor a objetos Productos con el
  # método eval para usarlo para completar la lista de productos
  for libro in lista_objetos:
      # Limpiando espacios en blanco para evitar errores
    libro = libro.strip()
    try:
      prod = eval(libro) # se obtiene el objeto producto
    except Exception as e:
      tmb.showerror("Error", f'{e}')
    else:
      # Añado todos los productos a una lista de productos_json
      lista_productos.append(prod)

  lista_objetos_pickle = pickle.dumps(lista_productos)

  # paso 6. Se borra el editor
  editor.delete('1.0','end')
  # paso 7. Se añade la lista_objetos_pickle al editor (objetos byte)
  editor.insert(f'1.0', lista_objetos_pickle)
  print(str(lista_objetos_pickle))
```

Resultado

En este caso particular, no hay ningún fichero con formato pickle, sino que es necesario un objeto Producto para transformarlo en pickle que lo codifica a texto* PICKLE (ojo que es binario) y para volver el texto Pickle a producto Producto.

- Pasando de objeto de la clase *Producto (de cualquier transformación de los formatos anteriores a objeto) a texto pickle* con el botón *obj→pickle.*

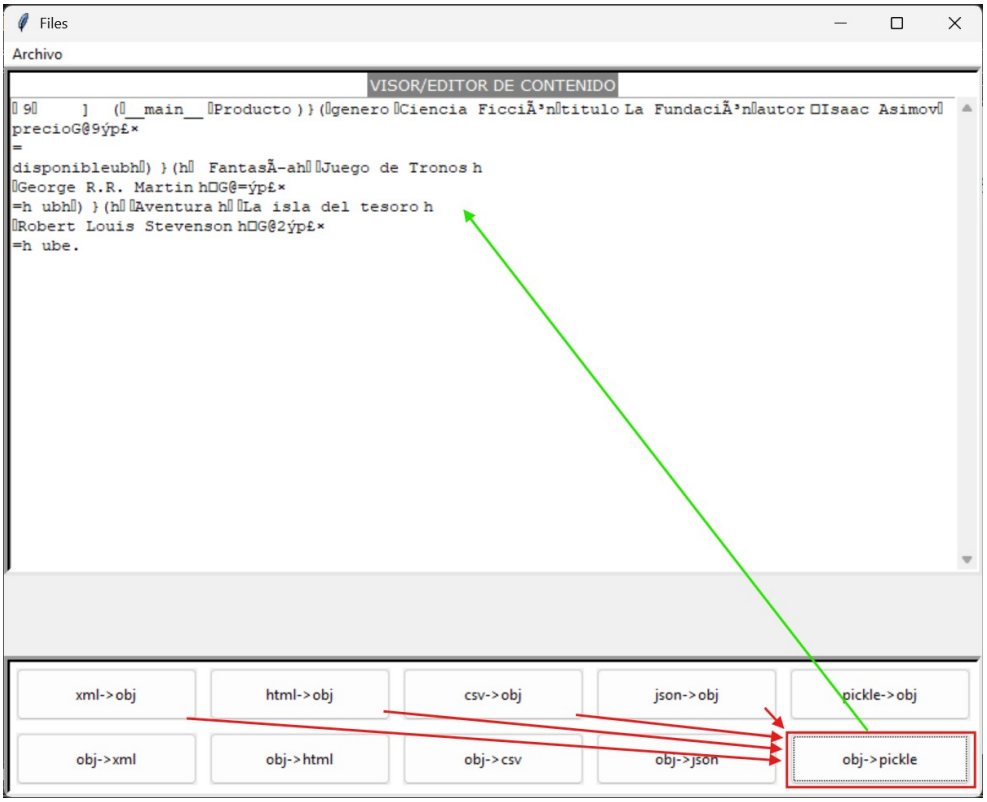


Figura 11: Mostrando pickler a partir del contendido de los objetos de la clase Producto.

- Pasando de texto pickle a objeto de la clase Producto con el botón pickle → obj. Por tanto, es necesario tener un texto pickle antes de utilizar este botón y ya reutilizarlo en cualquier formato.

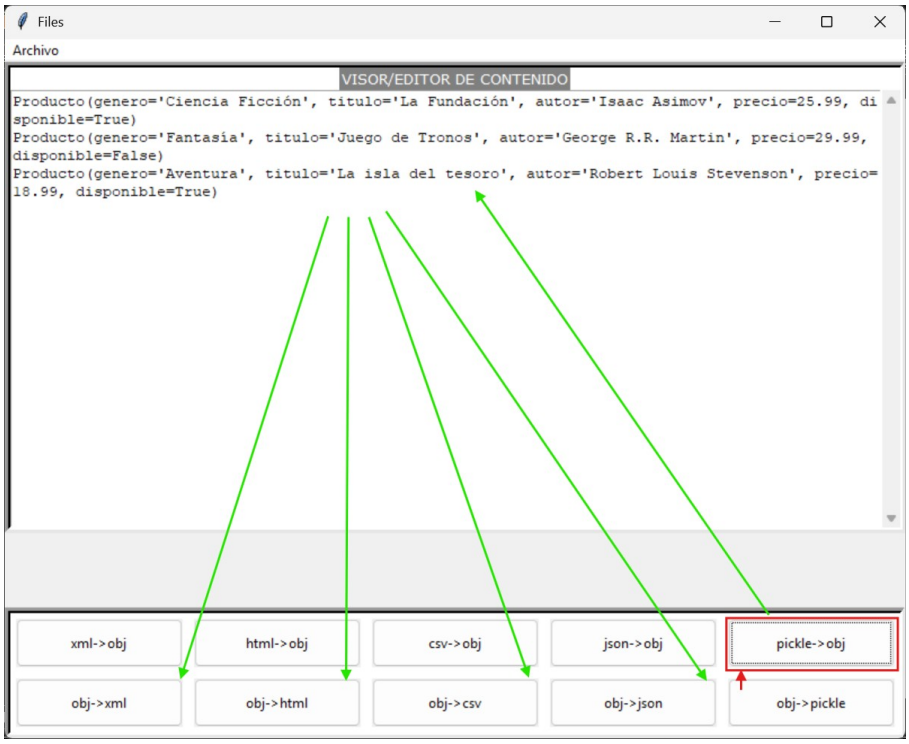


Figura 12: Mostrando objetos de la clase Productos a partir de un fichero csv

Aunque no se siga repitiendo el resto de acciones, se puede volver a realizar la fase anterior para volver a mostrar los objetos de la clase Producto con el botón *pickle → obj*.

Parte 6: Funcionalidad de los mensajes de error a través del formato mostrado en la UI de Tkinter.

Para mejorar la funcionalidad de los errores y mejorar la UX del usuario se ha realizado un pequeña modificación al código aportado por el profesor y se ha añadido una nueva función para comprobar los formatos antes de realizar las transformaciones en los diferentes formatos.

```
# paso 2. Se obtiene el texto del editor y se comprueba su formato
    texto_objetos = editor.get('1.0','end-1c') # 1c para evitar el \n final
    formato = detecta_formato_tkinter(texto_objetos)
    if not formato =='objeto Producto':
      tmb.showerror('ERROR DE FORMATO',f'Se detecta formato {formato.upper()}, no OBJETO
PRODUCTO')
      return # evita que recorra más código
```

Todas las funciones tienen el formato anterior, donde a partir de los datos mostrados en la UI se realiza una llamada a la función detecta_formato_tkinter() para comprobar si el formato mostrado es adecuado para cada uno de los botones de la UI.

A continuación se muestra la función que intenta diferenciar los formatos, hay que tener muy encuentra la posición de cada uno ya que, por ejemplo, el formato más común que puede dar confusión es el CSV, por ese motivo se añade al final del todo.

```
def detecta_formato_tkinter(str_editor):
  '''Analiza el contenido del editor y devuelve el formato detectado.

  Args:
    str_editor(str): contenido del editor tkinter

  Returns:
    Out (str): 'vacio', 'html', 'xml', 'json', 'pickle', 'objeto Producto', 'csv' o
'desconocido'
  '''
  # Paso 1: se comprueba si está vacío
  texto = str_editor.strip()
  if not texto: return "vacío"

  # Paso 2: se comprueba si es html (antes del xml ya que tienen formato parecido)
  # se recoge los primeros 200 caracteres minúscula para comprobar etiquetas
  inicio = texto.lower()[:200]
  # True si cualquiera (any) de las etiquetas html añadidas se encuentran en inicio
  if any(etiqueta in inicio for etiqueta in ['<html','<body','<div','!doctype html']):
    return "html"
```

```
  # Paso 3: se comprueba si es xml (después del html ya que se puede confundir)
  if texto.startswith('<?xml') or (texto.startswith('<') and texto.endswith('>')):
    try:
      ET.fromstring(texto) # es bastante estricto
      return 'xml'
    except ET.ParseError:
      pass # si no es xml continua comprobando el tipo de texto

  # Paso 4: se comprueba si es json, comprobando si empiezan/acaban con {} o con []
  if (texto.startswith('{') and texto.endswith('}')) or\
    (texto.startswith('[') and texto.endswith(']')):
    try:
      json.loads(texto)
      return "json"
    except json.JSONDecodeError:
      pass # si no es json continua comprobando el tipo de texto

  # Paso 5: se comprueba si es pickle(a través de formato hexadecimal/binario
  # o marcas de protocolo)
  if r'\x80\x03' in texto or '\x80\x04' in texto or texto.startswith('b'):
    return "pickle"

  # Paso 6: se comprueba si es un objeto Producto
  if (texto.startswith('Producto(')):
    return "objeto Producto"

  # Se añade al final debido a que se puede confundir con los formatos anteriores
  # Paso 7: se comprueba si es csv a través de líneas estucturadas con
comas/separadores
  lineas = [l for l in texto.split('\n') if l.strip()] # elimina las lineas vacías
  #
  if len(lineas)>0 and ',' in lineas[0]:
    return "csv"

  # Si no es xml, html, csv, json, pickle o vacío no es un formato conocido
  return "desconocido"
```

Resultado

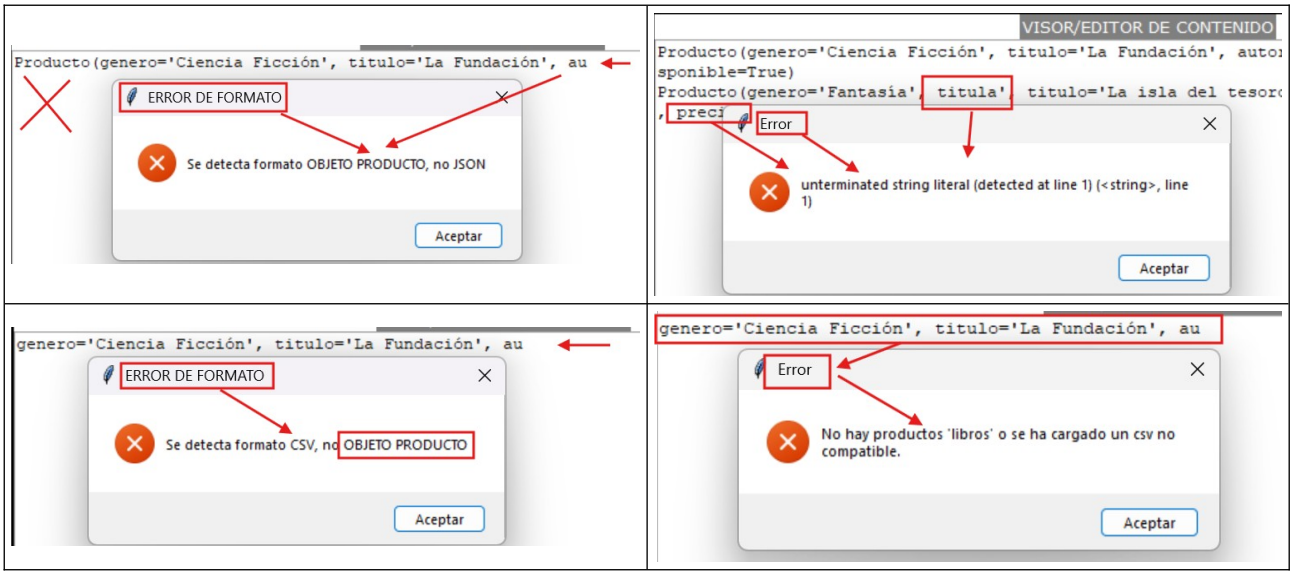
En este caso particular, se pretende probar los mensajes de error:


ERROR DE FORMATO
Se detecta formato VACÍO, no XML
Aceptar
xml->obj
html->obj
ERROR DE FORMATO
Se detecta formato VACÍO, no HTML
Aceptar
html->obj
ERROR DE FORMATO
Se detecta formato VACÍO, no CSV
Aceptar
csv->obj
ERROR DE FORMATO
Se detecta formato VACÍO, no OBJETO PRODUCTO
Aceptar
json->obj
pickle->obj
obj->json
obj->pickle
genero,titulo,autor,precio,disponible
Ciencia Ficción,La Fundación,Isaac Asimov,25.99,si
Fantasía,Juego de Tronos,George R.R. Martin,29.99,no
Aventura,La isla del tesoro,Robert Louis Stevenson,18.99,si
ERROR DE FORMATO
Se detecta formato CSV, no XML
Aceptar
<html>
<head>
<title>Libros de la librería</title>
ERROR DE FORMATO
Se detecta formato HTML, no CSV
Aceptar

[{
"genero": "Ciencia Ficci\u00f3n",
"titulo": "La Fundaci\u00f3n",
"autor": "Isaac Asimov",
ERROR DE FORMATO
Se detecta formato JSON, no HTML
Aceptar
<?xml version="1.0" encoding="UTF-8"?>
<libreria>
<libro genero="Ciencia Ficción">
<titulo>La Fundación</titulo>
ERROR DE FORMATO
Se detecta formato XML, no JSON
Aceptar
</libro>
[{
"genero": "Ciencia Ficci\u00f3n",
"titulo": "La Fundaci\u00f3n",
ERROR DE FORMATO
Se detecta formato JSON, no OBJETO PRODUCTO
Aceptar
<?xml version="1.0" encoding="UTF-8"?>
<libreria>
</libro>
</libreria>
ERROR DE FORMATO
Se detecta formato DESCONOCIDO, no XML
Aceptar
Producto(genero='Ciencia Ficción', titulo='La Fundación
sponible=True)
Producto(genero='Fantasía', titulo='Juego de Tronos',
ERROR DE FORMATO
Se detecta formato OBJETO PRODUCTO, no PICKLE
Aceptar
Archivo
VISOR/EDITOR DE CONTE
<html>
<head>
<title>
</h2>
La isla

Robert

18.99
Error
No hay libros o se ha cargado un html no compatible.
Aceptar



Como se puede observar en las imágenes anteriores, son errores que muestra la interfaz con la idea de facilitar la experiencia de usuario en caso de descuido. En este caso hay dos tipos de mensaje, uno generado por la función anterior y otra por el try-exception de cada función en caso de confundir los formatos (observar la imágenes finales). De esta manera queda muy protegido contra posibles errores que puedan crear inestabilidad de la aplicación.

Problemas encontrados, soluciones y/o sugerencias

En este caso, no ha sido sencillo pensar en como mostrar al usuario mensajes de error y diferenciarlos, aunque eso sí, para conseguirlo se ha creado una función. Su objetivo es detectar los tipos de formato y en caso de manipulación protegerse con el try-exception para informar en todo momento al usuario.

Conclusión

Unos casos prácticos muy útiles donde trabajar los conceptos introductorios de la unidad donde se aplica la información de los ejercicios de ejemplo de una manera muy didáctica y fluida con el objetivo de utilizarlo en la actividad obligatoria lo que reduce la frustración y mejora el aprendizaje.

Bibliografía/webgrafía

- Documentación del curso.
- Repositorio de GitHub compartido: https://github.com/oscgaresc/ADA